



Chitkara

PAPER OF CSE-2025 - PYTHON TEST10 EXAM

Q1. Courier company parcel fee update

A courier company is revising its parcel delivery charges due to recent changes in packaging material costs and fluctuating fuel prices. To maintain fair pricing while adjusting to operational expenses, the company decides to revise all parcel fees using the following rules:

- If a parcel fee is **below ₹100**, it should be **increased by 10%** to cover increased packaging and handling costs.
- If a parcel fee is **above ₹500**, it should be **reduced by 5%** to keep long-distance delivery charges competitive.
- After applying the adjustment, every updated fee must be rounded to the nearest integer.

Your task is to apply these rules to the parcel fee list and output the updated fees.

Input Format

A list of integers → parcel fees

Output Format

A list of updated parcel fees (integers)

Sample Input 1

[85, 620, 300, 40, 510, 95]

Sample Output 1

[94, 589, 300, 44, 484, 105]

Sample Input 2

[50, 100, 550, 75]

Sample Output 2

[55, 100, 522, 82]

ANSWER:

Test case 1 :

Input:

[85, 620, 300, 40, 510, 95]

Output:

[94, 589, 300, 44, 484, 105]

Test case 2 :

Input:

[50, 100, 550, 75]

Output:

[55, 100, 522, 82]

Test case 3 :

Input:

[99, 101, 501, 500]

Output:

[109, 101, 476, 500]

Test case 4 :

Input:

[80, 90, 95, 99]

Output:

[88, 99, 105, 109]

Test case 5 :

Input:

[600, 650, 700]

Output:

[570, 618, 665]

Test case 6 :

Input:

[100, 200, 300, 400, 500]

Output:

[100, 200, 300, 400, 500]

Q2. School Annual Event Ticket Pricing

Every year, a school organizes a grand cultural event that includes performances, exhibitions, and competitions. As part of its annual review, the school administration evaluates the ticket prices to make them affordable for students and parents while still covering basic event expenses. Due to changing decoration, stage setup, and logistics costs, the school decides to revise its event ticket prices using the rules below:

Price Revision Rules

- If a ticket price is **below ₹100**, it must be **increased by 10%** to cover basic event costs such as decorations and seating arrangements.
- If a ticket price **exceeds ₹500**, it must be **reduced by 5%** so that premium seating remains accessible to more visitors.
- After adjustment, each price must be rounded to the nearest integer.

Your task is to update the list of ticket prices according to these rules.

Input Format

A list of integers → ticket prices

Output Format

A list of updated prices (integers)

Sample Input 1

[60, 500, 510, 130]

Sample Output 1

[66, 500, 484, 130]

Sample Input 2

[80, 150, 999, 30, 510, 450]

Sample Output 2

[88, 150, 949, 33, 485, 450]

ANSWER:

Test case 1 :

Input:

[60, 500, 510, 130]

Output:

[66, 500, 484, 130]

Test case 2 :

Input:

[80, 150, 999, 30, 510, 450]

Output:

[88, 150, 949, 33, 484, 450]

Test case 3 :

Input:

[99, 101, 501, 500]

Output:

[109, 101, 476, 500]

Test case 4 :

Input:

[80, 90, 95, 99]

Output:

[88, 99, 105, 109]

Test case 5 :

Input:

[600, 650, 700]

Output:

[570, 618, 665]

Test case 6 :

Input:

[100, 200, 300, 400, 500]

Output:

[100, 200, 300, 400, 500]

Q3. Gym Membership Fee Revision

A gym decides to revise its membership fees due to increased costs of equipment servicing, facility maintenance, and staff upgrades. To maintain fairness and encourage more people to join, the gym announces new pricing rules.

Fee Update Rules

- **Membership fees below ₹100** → increase by **10%**, as the basic plans need adjustment due to higher utility costs.
- **Membership fees above ₹500** → decrease by **5%**, to keep premium plans competitive.
- All other fees remain **unchanged**.
- After updating, every fee must be **rounded to the nearest integer**.

Your Task

Take a list of membership fees and apply the above rules to generate the revised price list.

Input Format :

A list of integers → fee

Output Format :

A list of updated fee (integers)

Sample Input 1

[95, 600, 250, 80]

Sample Output 1

[105, 570, 250, 88]

Sample Input 2

[99, 101, 700, 10, 555, 350]

Sample Output 2

[109, 101, 665, 11, 527, 350]

ANSWER:

Test case 1 :

Input:

[95, 600, 250, 80]

Output:

[105, 570, 250, 88]

Test case 2 :

Input:

[99, 101, 700, 10, 555, 350]

Output:

[109, 101, 665, 11, 527, 350]

Test case 3 :

Input:

[99, 101, 501, 500]

Output:

[109, 101, 476, 500]

Test case 4 :

Input:

[80, 90, 95, 99]

Output:

[88, 99, 105, 109]

Test case 5 :

Input:

[600, 650, 700]

Output:

[570, 618, 665]

Test case 6 :

Input:

[100, 200, 300, 400, 500]

Output:

[100, 200, 300, 400, 500]

Q4. Scrolls of Nalanda Cipher Unlock

At the ruins of the ancient **Nalanda University**, archaeologists discover encrypted scrolls believed to be encoded using Chanakya's secret linguistic method. Every verse follows a strict vowel-consonant swap rule meant to hide the true teachings from invaders. You must decode these verses to restore the original wisdom.

Task

Use the following **bi-directional swapping rule** to decode the scroll message:

 $a \rightarrow b, b \rightarrow a$ $e \rightarrow f, f \rightarrow e$ $i \rightarrow j, j \rightarrow i$ $o \rightarrow p, p \rightarrow o$ $u \rightarrow v, v \rightarrow u$

Works case-insensitively and preserves original case.

Rules:

- If the character is a **vowel (a,e,i,o,u)** → convert it to the paired consonant.
- If it is the paired consonant (**b,f,j,p,v**) → convert it back to its vowel.
- Preserve original **case**.
- Leave all unrelated characters unchanged.

Input Format

A single encrypted string from the Nalanda scroll.

Output Format

Decoded (original) teaching as a string.

Sample Input

Knpwlfldgf arjngs opwfr

Sample Output

Knowledge brings power

ANSWER:

Test case 1 :

Input:

Knpwlfldgf arjngs opwfr

Output:

Knowledge brings power

Test case 2 :

Input:

Epcvs pn ypvv Gpbbs

Output:

Focus on your Goals

Test case 3 :

Input:

Hflp

Output:

Hello

Test case 4 :

Input:

Avrnjng Jslbnd

Output:

Burning Island

Test case 5 :

Input:

Mbrtjn Lvthbr

Output:

Martin Luthar

Test case 6 :

Input:

BFJPV

Output:

AEIOU

Q5. Scrolls of Hogwarts - The Cipher of Secrets

During a late-night exploration of the Restricted Section in the Hogwarts Library, Harry and Hermione discover a set of enchanted parchments. The texts are written in a strange cipher, rumored to be a secret spell invented by an ancient Hogwarts professor to hide powerful knowledge from dark wizards.

Each line on the parchment follows a strict **vowel–consonant swap charm**, making the real message unreadable to anyone who doesn't know the rule. Your job is to decode these magical messages and reveal the hidden wisdom.

Task

Use the following **bi-directional swapping rule** to decode the message:

 $a \rightarrow b, b \rightarrow a$ $e \rightarrow f, f \rightarrow e$ $i \rightarrow j, j \rightarrow i$ $o \rightarrow p, p \rightarrow o$ $u \rightarrow v, v \rightarrow u$

Works case-insensitively and preserves original case.

Rules:

- If the character is a **vowel (a,e,i,o,u)** → convert it to the paired consonant.
- If it is the paired consonant (**b,f,j,p,v**) → convert it back to its vowel.
- Preserve original **case**.
- Leave all unrelated characters unchanged.

Input Format

A single encrypted string from the Nalanda scroll.

Output Format

Decoded (original) teaching as a string.

Sample Input

Hfllp tp appk

Sample Output

Hello to book

ANSWER:

Test case 1 :

Input:

Hflfp tp appk

Output:

Hello to book

Test case 2 :

Input:

Epcvs pn ypvv Gpbbs

Output:

Focus on your Goals

Test case 3 :

Input:

Hflfp

Output:

Hello

Test case 4 :

Input:

Avrnjng Jslbnd

Output:

Burning Island

Test case 5 :

Input:

Mbrtjn Lvthbr

Output:

Martin Luthar

Test case 6 :

Input:

BFJPV

Output:

AEIOU

Q6. Wizard Wars Prophecy Decoder

During the great **Wizard Wars**, a secret council of witches and wizards hides its prophecies using an ancient cipher attributed to **Chanakya**. These prophecies speak of battles at **Hogwarts**, unavoidable

fates like in *Final Destination*, and the rise of heroes when darkness spreads. Each prophecy line is encoded using a strict **vowel–consonant swap rule**. Your task is to **decode** such an encoded prophecy and reveal the original message.

Task

Use the following **bi-directional swapping rule** to decode the prophecy message:

- $a \rightarrow b, b \rightarrow a$
- $e \rightarrow f, f \rightarrow e$
- $i \rightarrow j, j \rightarrow i$
- $o \rightarrow p, p \rightarrow o$
- $u \rightarrow v, v \rightarrow u$

Works case-insensitively and preserves original case.

Rules:

- If the character is a vowel (a, e, i, o, u) $\rightarrow$ convert it to the paired consonant.
- If it is the paired consonant (b, f, j, p, v) $\rightarrow$ convert it back to its vowel.
- Preserve original **case** (uppercase stays uppercase, lowercase stays lowercase).
- Leave all unrelated characters (other letters, spaces, punctuation) unchanged.

You are given one encoded prophecy string. Decode it to recover the original line.

Input Format

A single string representing the encoded wizard prophecy.

Output Format

A single string representing the decoded (original) prophecy.

Sample Input

Mbgjc bt Hpgwbrts sbufs ljufs

Sample Output

Magic at Hogwarts saves lives

ANSWER:

Test case 1 :

Input:

Knpwlfdfg arjngs opwfr

Output:

Knowledge brings power

Test case 2 :

Input:

Epcvs pn ypvf Gpbfs

Output:

Focus on your Goals

Test case 3 :

Input:

Hflfp

Output:

Hello

Test case 4 :

Input:

Avrnjng Jslnbd

Output:

Burning Island

Test case 5 :

Input:

Mbrtjn Lvthbr

Output:

Martin Luthar

Test case 6 :

Input:

BFJPV

Output:

AEIOU

Q7. Atlantis Coral Energy Pulse Evaluator

Deep within the submerged kingdom of Atlantis, researchers study the energy pulses emitted by ancient coral structures. These pulses help determine the health and stability of underwater ecosystems. However, due to water currents and bio-luminescent interference, some readings become corrupted. Each coral's pulse data is recorded as a tuple of values (integers or floats). But sometimes the scanners mistakenly capture invalid data like words or strange symbols. Your task is to clean these readings and identify which coral formations show stable pulse energy.

Your Task

Write a function **evaluate_coral_pulses(pulses: list) -> tuple** that:

- Takes a list of tuples, where each tuple includes pulse readings for one coral sample.
- Example: [(140, 130.5, 125), (200, "ERROR", 210), (80, 90, 100)]

For each tuple:

- Ignore invalid (non-numeric) values.
- Compute the average of valid readings.
- If the average is greater than **120**, the coral is considered “stable.”
- Collect the rounded integer averages of all stable corals and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try-except to handle:

- `TypeError` → if input is not a list → return "Invalid Data Type"
- `ValueError` → if the list is empty → return "No Pulse Data Found"

Input Format

A list of tuples containing numeric or mixed-type coral pulse readings

Output Format

A tuple of integers representing the rounded average pulse values of stable coral structures, or one of the error messages:

"Invalid Data Type"

"No Pulse Data Found"

Sample Input 1

```
[(140, 130.5, 125), (200, "ERROR", 210), (80, 90, 100)]
```

Sample Output 1

```
(132, 205)
```

Sample Input 2

```
[]
```

Sample Output 2

```
No Pulse Data Found
```

ANSWER:

Test case 1 :

Input:

```
[(140, 130.5, 125), (200, "ERROR", 210), (80, 90, 100)]
```

Output:

```
(132, 205)
```

Test case 2 :

Input:

```
[]
```

Output:

```
No Readings Found
```

Test case 3 :

Input:

```
"notalist"
```

Output:

Invalid Data Type

Test case 4 :

Input:

[(120, 121, 122), (90, 80, 70)]

Output:

(121,)

Test case 5 :

Input:

[(200, 250, "??", 300), (50, 60, "oops")]

Output:

(250,)

Test case 6 :

Input:

[(100, "bad", 110), (118.5, 119.5), (200, 201)]

Output:

(200,)

Q8. Cybersecurity Firewall Signal Integrity Checker

Within a high-security digital fortress, firewall systems constantly measure signal integrity values to detect stable and unstable data streams. But due to malware interference and corrupted packets, some signal readings are invalid. Each firewall's signal data is stored as a tuple of readings (integers or floats). At times, faulty sensors record invalid entries like strings or broken byte codes. Your task is to filter out corrupted values and identify which firewalls maintain stable signal integrity.

Your Task

Write a function **check_firewall_signals(signals: list) -> tuple** that:

- Takes a list of tuples, each containing signal integrity readings for a firewall module.
- Example: [(140, 130.5, 125), (200, "CORRUPT", 210), (80, 90, 100)]

For each tuple:

- Ignore non-numeric values.
- Compute the average of valid readings.
- If the average is **greater than 120**, the firewall is marked **"stable."**
- Collect the rounded integer averages of all stable firewalls and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try-except to handle:

- **TypeError** → if input is not a list → return "Invalid Data Type"
- **ValueError** → if the list is empty → return "No Signal Data Found"

Input Format

A list of tuples containing numeric or mixed-type firewall signal readings

Output Format

A tuple of integers representing the rounded average signal values of stable firewalls,
or one of the error messages:

"Invalid Data Type"

"No Signal Data Found"

Sample Input 1

```
[(140, 130.5, 125), (200, "CORRUPT", 210), (80, 90, 100)]
```

Sample Output 1

```
(132, 205)
```

Sample Input 2

```
[]
```

Sample Output 2

```
No Signal Data Found
```

ANSWER:

Test case 1 :

Input:

```
[(140, 130.5, 125), (200, "CORRUPT", 210), (80, 90, 100)]
```

Output:

```
(132, 205)
```

Test case 2 :

Input:

```
[]
```

Output:

```
No Readings Found
```

Test case 3 :

Input:

```
"notalist"
```

Output:

```
Invalid Data Type
```

Test case 4 :

Input:

```
[(120, 121, 122), (90, 80, 70)]
```

Output:
(121,)

Test case 5 :

Input:
[(200, 250, "??", 300), (50, 60, "oops")]
Output:
(250,)

Test case 6 :

Input:
[(100, "bad", 110), (118.5, 119.5), (200, 201)]
Output:
(200,)

Q9. Enchanted Forest Mana Stream Stabilizer

In the heart of the Enchanted Forest, druids measure the mana flow from magical trees to ensure the balance of the forest's energy. However, due to wandering spirits and unpredictable mana surges, some readings get corrupted. Each tree's mana stream data is stored as a tuple of readings (integers or floats). But sometimes, magical interference causes invalid entries like words or mystical symbols. Your task is to clean these readings and identify which trees maintain stable mana flow.

Your Task

Write a function **stabilize_mana_streams(streams: list) -> tuple** that:

- Accepts a list of tuples, each containing mana readings for one magical tree.
- Example: `[(150, 160, 170), (100, 110, "GLOW"), (200.5, 199.5, 210)]`

For each tuple:

- Ignore invalid (non-numeric) values.
- Compute the average of valid readings.
- If the average is greater than 120, the tree is considered "stable."
- Collect the rounded integer averages of all stable trees and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try-except to handle:

- **TypeError** → if input is not a list → return "Invalid Data Type"
- **ValueError** → if the list is empty → return "No Mana Data Found"

Input Format

A list of tuples containing numeric or mixed-type mana stream readings

Output Format

A tuple of integers representing the rounded average mana values of stable trees,
or one of the error messages:
"Invalid Data Type"
"No Mana Data Found"

Sample Input 1

```
[(150, 160, 170), (100, 110, "GLOW"), (200.5, 199.5, 210)]
```

Sample Output 1

```
(160, 203)
```

Sample Input 2

```
"NOT_A_LIST"
```

Sample Output 2

```
Invalid Data Type
```

ANSWER:

Test case 1 :

Input:

```
[(150, 160, 170), (100, 110, "GLOW"), (200.5, 199.5, 210)]
```

Output:

```
(160, 203)
```

Test case 2 :

Input:

```
[]
```

Output:

```
No Readings Found
```

Test case 3 :

Input:

```
"notalist"
```

Output:

```
Invalid Data Type
```

Test case 4 :

Input:

```
[(120, 121, 122), (90, 80, 70)]
```

Output:

```
(121,)
```

Test case 5 :

Input:

```
[(200, 250, "??", 300), (50, 60, "oops")]
```

Output:

```
(250,)
```

Test case 6 :

Input:

```
[(100, "bad", 110), (118.5, 119.5), (200, 201)]
```

Output:

```
(200,)
```